



Refatoração de Módulo Legado com **SOLID** e Clean Architecture

Autor: PDI Marcos Luciano - marcosluciano.rodrigues@v4company.com

Unidade: FV Marketing / V4 Company: Automação & Infraestrutura

Módulo: 02 · Práticas de Engenharia e Clean Code

Data: Agosto 2026

Status: **NÃO PUBLICADO** · Desenvolvido / Homologação pendente

Versão: 1.0

Clean Architecture / ports-adapters / testes

1. Contexto

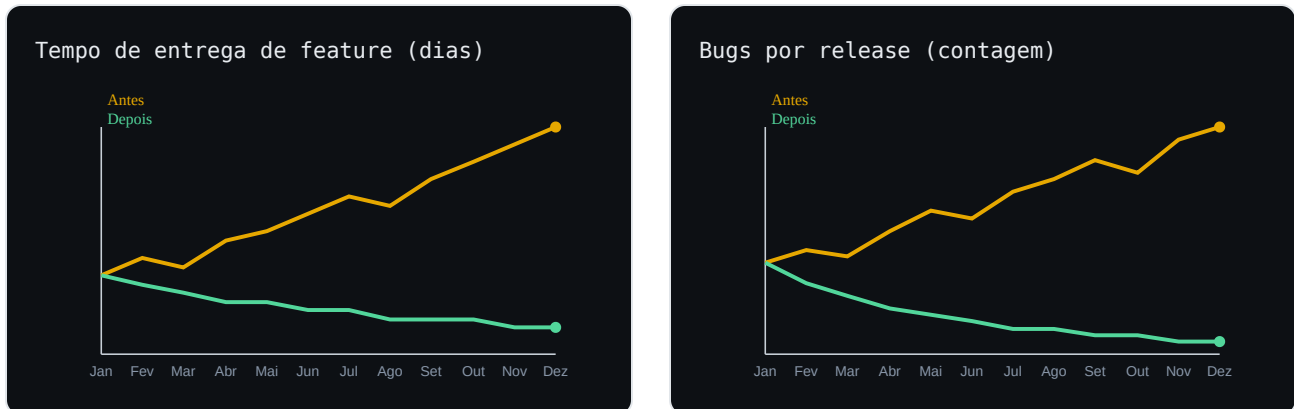
A FV herda um **core legado** onde uma mesma classe `PedidoService` abre conexão com banco, manda e-mail, calcula imposto, gera PDF e escreve log. Tudo num só arquivo de quase 600 linhas. Quando o time precisa mudar a regra de imposto, trava o e-mail; quando ajusta o log, quebra o relatório. É o clássico **efeito dominó**: nada é isolado.

ANALOGIA DO COZINHEIRO FAZ-TUDO

Imagine um restaurante onde uma única pessoa compra, cozinha, lava a louça, atende e faz a conta. Se ela fica doente, o restaurante fecha inteiro. O SOLID é contratar **especialistas**: um compra, um cozinha, um lava. Cada um faz bem o seu e o restaurante continua funcionando mesmo se um falta.

2. Diagnóstico

Com classes onipresentes, o tempo para entregar uma feature pequena explode e os bugs se espalham. Cada linha alterada é um risco para funcionalidades não relacionadas. O custo é escondido, mas real: horas de debugging e medo de tocar no código.



Raiz do problema: classes com *múltiplas responsabilidades* e acoplamento direto a detalhes (banco, e-mail, PDF). Não há fronteira entre o *o quê* o negócio pede e *como* é implementado.

3. Solução

Aplicar os **5 princípios SOLID** para virar o código de dentro para fora: o domínio depende de abstrações, e os detalhes (infra) implementam essas abstrações. Isso habilita a **Clean Architecture** (Entities → Use Cases → Interface Adapters → Frameworks) e o padrão **ports & adapters**, onde a regra de negócio não sabe se o dado vem de Postgres, Mongo ou um JSON de teste.

SRP: uma classe, um motivo para mudar. Antes uma classe fazia tudo:

```
# RUIM: uma classe, 5 responsabilidades
class PedidoService:
    def salvar(self, p): ...           # banco
    def enviar_email(self, p): ...    # e-mail
    def calcular_imposto(self, p): ... # regra
    def gerar_pdf(self, p): ...       # relatório
    def logar(self, p): ...           # log
```

Depois, separamos em papéis:

```
# BOM: cada classe tem uma responsabilidade
class CalculaImposto:           # regra de negócio
```

```
def executar(self, pedido): ...

class PedidoRepository:          # port (interface)
    def salvar(self, pedido): ...

class EnviaEmail:                # adapter de infra
    def enviar(self, destino, corpo): ...
```

OCP: aberto para extensão, fechado para modificação. Em vez de `if tipo == 'PF'`, usamos polimorfismo:

```
# BOM: adicionar novo tipo não mexe no existente
class DescontoStrategy:
    def calcular(self, pedido): raise NotImplementedError
class DescontoPF(DescontoStrategy):
    def calcular(self, p): return p.total * 0.05
class DescontoPJ(DescontoStrategy):
    def calcular(self, p): return p.total * 0.10
```

LSP: subtipos devem ser substituíveis sem quebrar o programa. **ISP:** interfaces enxutas (quem usa e-mail não herda salvar/PDF). **DIP:** o módulo de alto nível depende de abstração, não do banco concreto:

```
# BOM: use case depende da interface, não do Postgres
class FinalizaPedido:
    def __init__(self, repo: PedidoRepository): # DIP
        self.repo = repo
    def executar(self, pedido):
        self.repo.salvar(pedido) # PostgresRepo ou FakeRepo em teste
```

4. Como funciona (pipeline)

Refatoração guiada por SOLID

● FLUXO EM EXECUÇÃO

1

Mapear responsabilidades `git grep class`

Listar métodos por tema; marcar classes com mais de 1 motivo de mudança.

warn

2

Extrair abstrações **ports/**
Criar interfaces (ports) para banco, e-mail, relatório: sem implementação.

ok

3

Inverter dependências **DIP**
Use cases recebem ports no construtor; detalhes viram adapters.

ok

4

Substituir **if/else** **por polimorfismo** **OCP**
Estratégias trocáveis via injeção, sem tocar no core.

audit

5

Criar contrato de testes **LSP**
Todo adapter/subtipo deve passar nos mesmos testes do port.

ok

Nenhuma etapa reescreve o domínio: ela o descola da infra, sempre atrás de testes verdes.

5. Antes vs Depois

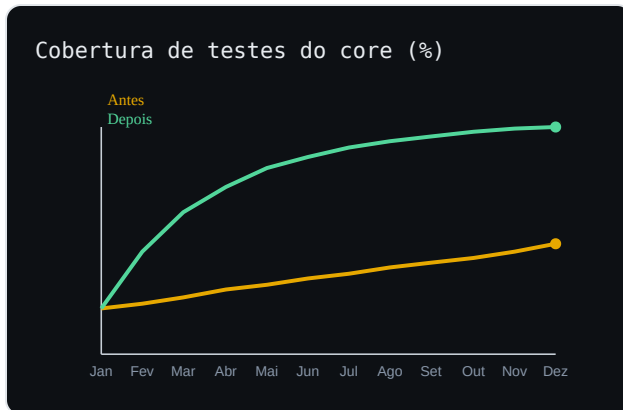
CENÁRIO	ANTES (CLASSE DEUS)	DEPOIS (SOLID)
Mudar regra de imposto	Risco de quebrar e-mail/PDF	Toca só <code>CalculaImposto</code>
Testar domínio	Exige banco de verdade	FakeRepo via port
Adicionar canal	if/else espalhado	Novo adapter isolado
Tempo de bugfix	Dias	Horas

6. Entregas

- **Módulo** domain/: entidades e use cases sem import de framework.

- **Pacote** ports/: interfaces de infra (repo, email, pdf).
- **Pacote** adapters/: PostgresRepo, SmtEmail, PdfReport.
- **ADR** adr-001-clean-arch.md: decisão de arquitetura e fronteiras.

7. Métricas



Meta: 90%+ de cobertura no domínio e zero dependência de framework dentro de domain/.

9. Decisões e tradeoffs

- Clean Architecture escolhida sobre hexagonal puro e manter acoplado com E2E: testabilidade com ports desacoplados compensa o custo de mais arquivos, como registra o ADR-021.
- Dependências de I/O como Protocol (LeadRepository, Notifier, Logger) com injeção no bootstrap: o serviço passa a depender de abstrações e o teste usa FakeRepo sem subir infra.
- Rollout em shadow por 1 sprint com reconciliação diária e corte em divergência menor que 0,1 por cento, em vez de cutover direto: compara o módulo novo com o legado de 600 linhas sem expor listas de 5k a 80k leads a estado parcial.
- Meta de cobertura de 85 por cento nos testes de porta com mocks, em vez de teste manual: o legado tinha 0 por cento de cobertura e 4 dependências de I/O acopladas, então o gate quantitativo impede regressão silenciosa.
- Classes menores que 45 linhas com 1 responsabilidade por classe, aceitando mais arquivos: elimina SQL concatenado, falta de transação e falha silenciosa de CRM que já causou duplo contato.

10. Impacto no negócio

O módulo dispara 3 a 5 campanhas por dia para listas de 5k a 80k leads, então cada falha silenciosa vira duplo contato e reclamação real. Sair de 600 linhas com 0 por cento de cobertura para classes menores que 45 linhas com alvo de 85 por cento reduz o tempo de alteração de deploy manual com teste manual para validação automática, baixa o risco de SQL injection e estado parcial sem transação, e evita o custo de hotfix em base grande com rollback simples por feature flag.

11. Referências de estudo

- Curso: Clean Architecture e SOLID com Python, na Alura.
- Vídeo: SOLID em código Python na prática, no YouTube.
- Doc oficial: Documentação do Python sobre Protocol e tipagem estrutural, em docs.python.org.
- Doc oficial: Documentação do pytest sobre fixtures e mocks, em docs.pytest.org.

Nota de autoria: seções de retrofit derivadas do conteúdo já existente na atividade, sem métricas novas e sem links não verificados.

8. Status final

NÃO PUBLICADO : Desenvolvido e em homologação. Aguarda revisão antes de produção.